Running the proofs

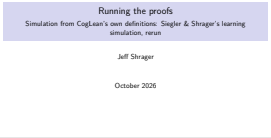
Simulation from CogLean's own definitions: Siegler & Shrager's learning simulation, rerun

Jeff Shrager

October 2026

Running the proofs

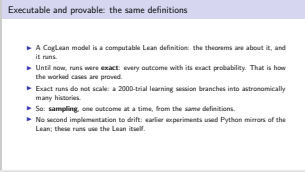
2026-10-30



Everything so far in CogLean has been proof: theorems about models, and exact computations. But Lean is a programming language too, and the definitions we prove things about are programs. This talk shows them running as simulations: many trials, with learning between them. The first target is the learning simulation from Siegler and Shrager's 1984 chapter, rerun from the Lean model and compared with the chapter's own figure.

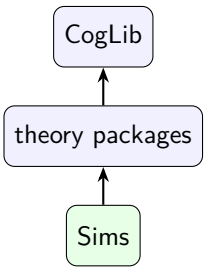
Executable and provable: the same definitions

- A CogLean model is a computable Lean definition: the theorems are about it, and it runs.
- Until now, runs were **exact**: every outcome with its exact probability. That is how the worked cases are proved.
- Exact runs do not scale: a 2000-trial learning session branches into astronomically many histories.
- So: **sampling**, one outcome at a time, from the *same* definitions.
- No second implementation to drift: earlier experiments used Python mirrors of the Lean; these runs use the Lean itself.

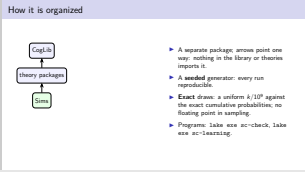


A CogLean model is a computable Lean definition, so the same code the theorems are about can be run. Until now, running meant computing exactly: every outcome with its exact probability, which is how worked cases become theorems. But exact runs don't scale to learning: two thousand trials, each with probabilistic outcomes that change what's learned, branch into astronomically many histories. So we sample, one outcome at a time, from the same definitions. That matters: earlier experiments were prototyped in Python mirrors of the Lean, which can drift from it. Here there's only one implementation.

How it is organized



- A separate package; arrows point one way: nothing in the library or theories imports it.
- A **seeded** generator: every run reproducible.
- **Exact** draws: a uniform $k/10^9$ against the exact cumulative probabilities; no floating point in sampling.
- Programs: `lake exe sc-check`, `lake exe sc-learning`.



The simulations live in their own package, Sims, which depends on the library and the theories, while nothing depends on it, so the proofs, the theory matrix and the catalog never see simulation code. The generator is seeded, so every run is reproducible. And the draws are exact: a uniform rational is compared with the model's exact cumulative probabilities, with no floating point in sampling at all. There are two programs so far: one checks the sampler, one runs the learning simulation.

First, check the sampler against the model

- A trial is drawn stage by stage, using the model's own retrieval, elaboration and learning; counting object by object.
- On small cases, compare every outcome's frequency with its *exact* probability under the model's `trial`:

Case (40,000 draws each)	worst $\lvert z \rvert$	within 4σ
$1+1$, search length 1, criterion .3	1.30	yes
$1+2$, search length 2, criterion .2	1.60	yes
$2+2$, search length 3, criterion .5	1.25	yes

So the simulation draws from the model, not from an approximation of it.

Running the proofs

First, check the sampler against the model

First, check the sampler against the model

- A trial is drawn stage by stage, using the model's own retrieval, elaboration and learning; counting object by object.
- On small cases, compare every outcome's frequency with its *exact* probability under the model's `trial`:

Case (40,000 draws each)	worst $\lvert z \rvert$	within 4σ
$1 + 1$, search length 1, criterion .3	1.30	yes
$1 + 2$, search length 2, criterion .2	1.60	yes
$2 + 2$, search length 3, criterion .5	1.25	yes

So the simulation draws from the model, not from an approximation of it.

Before trusting a long run, check the sampler. A trial is drawn stage by stage, using the model's own retrieval, elaboration and strengthening; only counting is drawn object by object, because the model's full counting distribution is exponential in the sum. Then, on small cases where the exact distribution is easy to compute, every outcome's sampled frequency is compared with its exact probability under the model's trial. All are well within four standard errors, the worst at one point six. So the simulation draws from the model itself.

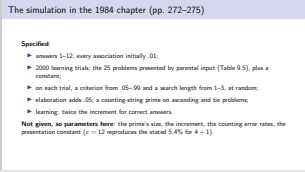
The simulation in the 1984 chapter (pp. 272–275)

Specified:

- answers 1–12, every association initially .01;
- 2000 learning trials; the 25 problems presented by parental input (Table 9.5), plus a constant;
- on each trial, a criterion from .05–.99 and a search length from 1–3, at random;
- elaboration adds .05; a counting-string prime on ascending and tie problems;
- learning: twice the increment for correct answers.

Not given, so parameters here: the prime's size, the increment, the counting error rates, the presentation constant ($c = 12$ reproduces the stated 5.4% for $4 + 1$).

The simulation in the 1984 chapter (pp. 272–275)



Here's what the 1984 chapter specifies about its simulation: twelve possible answers, each starting with a weak association; two thousand learning trials, with problems presented in proportion to how often parents presented them; a random confidence criterion and search length on each trial; elaboration; a prime from the counting string; and twice the learning increment for correct answers. Some numbers aren't given: the size of the prime, the increment, the counting error rates, and the constant added to the presentation frequencies. Those become parameters, and a constant of twelve reproduces the frequencies the chapter does state.

Searching the unspecified parameters

Mean |difference| from Fig. 9.8's peaks and shares (10 runs each):

increment \ prime	.01	.02	.05	.1	.2
1/400	.18	.18	.18	.18	.18
1/250	.11	.10	.10	.09	.08
1/200	.10	.09	.09	.08	**.06**
1/150	.18	.17	.18	.17	.14
1/100	.43	.42	.44	.41	.40

An increment near 1/200 fits; faster learning overshoots. The fit improves with a *larger* prime.

Running the proofs

Searching the unspecified parameters

Since some parameters aren't given, I searched them coarsely, comparing with the peaks and shares in the chapter's Figure 9.8. An increment around one two-hundredth fits best; faster learning overshoots badly, slower undershoots. And the fit keeps improving as the prime gets larger. This is a coarse sweep, not a fit, and I'll come back to what it suggests about the prime.

Searching the unspecified parameters

Mean |difference| from Fig. 9.8's peaks and shares (10 runs each):

increment \ prime	.01	.02	.05	.1	.2
1/400	.18	.18	.18	.18	.18
1/250	.11	.10	.10	.09	.08
1/200	.10	.09	.09	.08	**.06**
1/150	.18	.17	.18	.17	.14
1/100	.43	.42	.44	.41	.40

An increment near 1/200 fits; faster learning overshoots. The fit improves with a *larger* prime.

Against Fig. 9.8, after 2000 trials

Problems (effect)	Peak	Chapter	Share	Chapter
$1+4$ vs $2+3$ (counting string)	.54 vs .40	.67 vs .37	.75 vs .61	.77 vs .46
$4+1$ vs $3+2$ (frequency)	.85 vs .55	.80 vs .51	.78 vs .71	.76 vs .69
$3+4$ vs $4+5$ (sum)	.31 vs .32	.39 vs .32	.55 vs .52	.50 vs .43

Increment $1/200$, prime .2, counting errors .05; means over 20 runs. Peak: strength at the correct answer. Share: its fraction of the total.

Running the proofs

└Against Fig. 9.8, after 2000 trials

Against Fig. 9.8, after 2000 trials

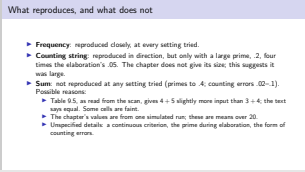
Problems (effect)	Peak	Chapter	Share	Chapter
$1+4$ vs $2+3$ (counting string)	.54 vs .40	.67 vs .37	.75 vs .61	.77 vs .46
$4+1$ vs $3+2$ (frequency)	.85 vs .55	.80 vs .51	.78 vs .71	.76 vs .69
$3+4$ vs $4+5$ (sum)	.31 vs .32	.39 vs .32	.55 vs .52	.50 vs .43

Increment 1/200, prime .2, counting errors .05; means over 20 runs. Peak: strength at the correct answer. Share: its fraction of the total.

Here are the results at the best setting, against the chapter's three comparisons. Frequency of presentation: four plus one against three plus two, eighty-five against fifty-five, close to the chapter's eighty against fifty-one. The counting string: one plus four against two plus three, in the right direction, though the gap in peaks is smaller than the chapter's. And the sum of the addends: three plus four against four plus five, essentially tied, where the chapter shows a clear advantage for the smaller sum.

What reproduces, and what does not

- **Frequency**: reproduced closely, at every setting tried.
- **Counting string**: reproduced in direction, but only with a large prime, .2, four times the elaboration's .05. The chapter does not give its size; this suggests it was large.
- **Sum**: not reproduced at any setting tried (primes to .4; counting errors .02–.1). Possible reasons:
  - Table 9.5, as read from the scan, gives $4 + 5$ slightly more input than $3 + 4$; the text says equal. Some cells are faint.
  - The chapter's values are from one simulated run; these are means over 20.
  - Unspecified details: a continuous criterion, the prime during elaboration, the form of counting errors.

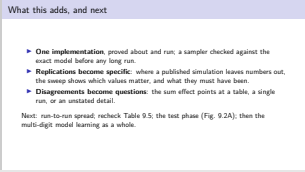


So: the frequency effect comes out closely everywhere. The counting-string effect comes out in direction, but only if the prime is large, four times the size of elaboration; the chapter never gave the prime's size, so this is a small inference about the original. And the sum effect doesn't come out at any setting I tried. There are three candidate reasons. My reading of the parental-input table from the scan gives four plus five slightly more input than three plus four, where the text says they're equal, and some cells are faint. The chapter's numbers are from a single simulated run, while mine are averages. And there are details the chapter doesn't specify. Each of these is checkable.

What this adds, and next

- **One implementation**, proved about and run; a sampler checked against the exact model before any long run.
- **Replications become specific**: where a published simulation leaves numbers out, the sweep shows which values matter, and what they must have been.
- **Disagreements become questions**: the sum effect points at a table, a single run, or an unstated detail.

Next: run-to-run spread; recheck Table 9.5; the test phase (Fig. 9.2A); then the multi-digit model learning as a whole.



What does this add? One implementation that is both proved about and run, with the sampler checked against the exact model before any long run is trusted. Replications become specific: where a published simulation leaves numbers out, a sweep shows which ones matter and what they must have been, like the size of the prime. And disagreements become concrete questions, here about a table, a single run, or an unstated detail. Next: the spread across runs, a clean copy of the parental-input table, the chapter's test phase, and then the multi-digit model learning as a whole, recognition, associations and problem solving together.